

Documento de Anexos Técnicos

Trabajo de Fin de Máster en Data Science, Big Data & AI

Universidad Complutense de Madrid

Título: Sistema Inteligente de Dynamic Pricing y Optimización de Capacidad en el Transporte Interurbano mediante Deep Reinforcement Learning

Autora: Inmaculada Osuna Muñoz

17 de septiembre de 2026

Índice

Preámbulo	2
1. Anexo A: Configuración del Entorno e Importación de Librerías	3
2. Anexo B: Ingesta y ETL Masivo (PySpark)	4
3. Anexo C: Entorno de Simulación (Gymnasium) y DP	7
4. Anexo D: Baseline Machine Learning Tradicional	10
5. Anexo E: Entrenamiento del Agente Autónomo PPO	12
6. Anexo F: Evaluación de Negocio y Simulación Monte Carlo	14
7. Anexo G: Reconstrucción e Interpretación Económica PPO	16
8. Anexo H: MLOps y Exportación Serializada a Web	17
9. Anexo I: Arquitectura Guardrails y Alertas en Vanilla JS	18

Preámbulo

Este documento recopila el código fuente que sustenta el desarrollo de la Memoria del Trabajo de Fin de Máster.

Adicionalmente, se facilitan a continuación los enlaces a los recursos externos del proyecto:

- **Repositorio (GitHub):** [Enlace al repositorio](#)
- **Vídeo de Presentación :** [Enlace al vídeo](#)
- **Aplicación Web:** [Enlace a la aplicación web](#)

Asimismo, se han concedido los permisos necesarios para la visualización del repositorio privado.

Todas las celdas de procesamiento, entrenamiento y evaluación se exponen en su totalidad en los siguientes apartados.

1. Anexo A: Configuración del Entorno e Importación de Librerías

Bloque inicial para la importación de dependencias analíticas, definición de constantes del negocio ferroviario e inicialización de semillas estocásticas para asegurar la trazabilidad.

```

1      import json
2      import os
3      import random
4      import sys
5      import warnings
6      from pathlib import Path
7
8      import gymnasium as gym
9      import matplotlib.pyplot as plt
10     import numpy as np
11     import pandas as pd
12     import pyspark
13     import sklearn
14     import stable_baselines3
15     from gymnasium import spaces
16     from scipy.stats import poisson
17     from sklearn.ensemble import RandomForestRegressor
18     from stable_baselines3 import PPO
19     from stable_baselines3.common.callbacks import EvalCallback
20     from stable_baselines3.common.env_checker import check_env
21     from stable_baselines3.common.monitor import Monitor
22     from stable_baselines3.common.utils import set_random_seed
23
24     from pyspark.sql import SparkSession
25     from pyspark.sql.functions import (
26         avg, col, count, countDistinct, datediff, expr, lit,
27         round as spark_round, to_date
28     )
29
30     SEED = 42
31     MAX_DIAS = 30
32     CAPACIDAD = 100
33
34     ELASTICIDAD_NOMINAL = -1.2
35     FACTOR_DEMANDA_NOMINAL = 1.0
36     PENALIZACION_VACIOS = 0.10
37
38     random.seed(SEED)
39     np.random.seed(SEED)
40     set_random_seed(SEED)
41
42     BASE_DIR = Path.cwd()
43     RUTA_CSV = BASE_DIR / "thegurus-opendata-renfe-trips.csv"
44
45     if not RUTA_CSV.exists():
46         raise FileNotFoundError(f"No se encuentra el dataset en: {RUTA_CSV}")
47
48     warnings.filterwarnings("ignore", category=UserWarning, module="gymnasium")
49     warnings.filterwarnings("ignore", category=DeprecationWarning, module="pyspark")
50
51     print("=" * 60)
52     print("CONFIGURACIÓN DEL ENTORNO")
53     print("=" * 60)
54     print(f"Python:           {sys.version.split()[0]}")
55     print(f"NumPy:             {np.__version__}")
56     print(f"Pandas:            {pd.__version__}")
57     print(f"Scikit-learn:      {sklearn.__version__}")
58     print(f"PySpark:           {pyspark.__version__}")
59     print(f"Gymnasium:         {gym.__version__}")
60     print(f"Stable-Baselines3: {stable_baselines3.__version__}")
61

```

Listing 1: Importación y variables globales

2. Anexo B: Ingesta y ETL Masivo (PySpark)

Pipeline de computación distribuida para la lectura, limpieza y particionado de millones de registros, garantizando matemáticamente la prevención del *Data Leakage*.

```

1      # Definición de columnas
2      COL_FECHA_COMPRA = "insert_date"
3      COL_FECHA_SALIDA = "departure"
4      COL_PRECIO = "price"
5      COL_ORIGEN = "origin"
6      COL_DESTINO = "destination"
7      COL_TIPO_VEHICULO = "vehicle_type"
8      COL_CLASE = "vehicle_class"
9      COL_TARIFA = "fare"
10
11     # Sesión Spark
12     spark = SparkSession.builder.appName("TFM_Renfe_DynamicPricing_DRL").getOrCreate()
13
14     df_raw = (
15         spark.read.format("csv")
16         .option("header", "true")
17         .option("inferSchema", "true")
18         .option("quote", "'")
19         .option("escape", "'')
20         .option("multiline", "true")
21         .load(str(RUTA_CSV))
22     )
23
24     from pyspark.sql.functions import translate, upper
25
26     df_clean = (
27         df_raw
28         .withColumn(COL_ORIGEN, translate(upper(col(COL_ORIGEN)), 'ÁÉÍÓÚ', 'AEIOU'))
29         .withColumn(COL_DESTINO, translate(upper(col(COL_DESTINO)), 'ÁÉÍÓÚ', 'AEIOU'))
30         .withColumn("fecha_compra_dt", to_date(col(COL_FECHA_COMPRA)))
31         .withColumn("fecha_salida_dt", to_date(col(COL_FECHA_SALIDA)))
32         .withColumn("lead_time_dias", datediff(col("fecha_salida_dt"), col("
33 fecha_compra_dt")))
34         .filter(col("lead_time_dias").between(0, MAX_DIAS))
35         .filter(col(COL_PRECIO).isNotNull() & (col(COL_PRECIO) > 0))
36         .dropna(subset=[COL_ORIGEN, COL_DESTINO, COL_TIPO_VEHICULO, COL_CLASE, COL_TARIFA
37 , "fecha_salida_dt", "fecha_compra_dt"])
38         .cache()
39     )
40
41     fechas_salida = [
42         fila["fecha_salida_dt"] for fila in (
43             df_clean.select("fecha_salida_dt")
44             .where(col("fecha_salida_dt").isNotNull())
45             .distinct()
46             .orderBy("fecha_salida_dt")
47             .collect()
48         )
49     ]
50
51     idx_fin_train = max(1, int(0.70 * len(fechas_salida)))
52     idx_fin_val = min(max(idx_fin_train + 1, int(0.80 * len(fechas_salida))), len(
53 fechas_salida) - 1)
54
55     fecha_corte_train = fechas_salida[idx_fin_train]
56     fecha_corte_test = fechas_salida[idx_fin_val]
57
58     df_train_raw = df_clean.filter(col("fecha_salida_dt") < lit(fecha_corte_train))
59     df_val_raw = df_clean.filter((col("fecha_salida_dt") >= lit(fecha_corte_train)) &
60 (col("fecha_salida_dt") < lit(fecha_corte_test)))
61     df_test_raw = df_clean.filter(col("fecha_salida_dt") >= lit(fecha_corte_test))
62
63     CLAVES_SERVICIO = [COL_ORIGEN, COL_DESTINO, "fecha_salida_dt"]
64
65     train_servicios = df_train_raw.select(*CLAVES_SERVICIO).dropDuplicates()
66     val_servicios = df_val_raw.select(*CLAVES_SERVICIO).dropDuplicates()

```

```

63     test_servicios = df_test_raw.select(*CLAVES_SERVICIO).dropDuplicates()
64
65     solapamiento_train_val = train_servicios.intersect(val_servicios).count()
66     solapamiento_train_test = train_servicios.intersect(test_servicios).count()
67     solapamiento_val_test = val_servicios.intersect(test_servicios).count()
68
69     if (solapamiento_train_val == 0 and solapamiento_train_test == 0 and
solapamiento_val_test == 0):
70         print("OK: no existe solapamiento de servicios entre las particiones.")
71     else:
72         raise ValueError("Sigue existiendo solapamiento de servicios.")
73
74     def agregar_precios(df):
75         return (
76             df.groupBy(COL_ORIGEN, COL_DESTINO, "lead_time_dias", COL_TIPO_VEHICULO,
COL_CLASE, COL_TARIFA)
77             .agg(
78                 count("*").alias("n_observacionesPrecio"),
79                 spark_round(avg(COL_PRECIO), 2).alias("precio_medio"),
80                 expr(f"percentile_approx({COL_PRECIO}, 0.5)").alias("precio_mediano"),
81                 spark_round(expr(f"stddev({COL_PRECIO})"), 2).alias("precio_std"),
82                 expr(f"max({COL_PRECIO})").alias("precio_maximo"),
83             )
84         )
85
86     df_precios_train_pd = agregar_precios(df_train_raw).toPandas()
87     df_precios_val_pd = agregar_precios(df_val_raw).toPandas()
88     df_precios_test_pd = agregar_precios(df_test_raw).toPandas()
89
90     def crear_referencia_precio(df, origen="MADRID", destino="BARCELONA"):
91         ruta = df[(df[COL_ORIGEN] == origen) & (df[COL_DESTINO] == destino)].copy()
92         tarifa_maxima_historica = float(ruta["precio_maximo"].max())
93         ruta["precio_ponderado"] = ruta["precio_mediano"] * ruta["n_observaciones_precio"
]
94         referencia = ruta.groupby("lead_time_dias")[["precio_ponderado", "
n_observaciones_precio"]].sum()
95         referencia["precio_base"] = referencia["precio_ponderado"] / referencia["
n_observaciones_precio"]
96         referencia = referencia.reset_index()[["lead_time_dias", "precio_base"]]
97         dias_completos = pd.DataFrame({"lead_time_dias": np.arange(0, MAX_DIAS + 1)})
98         referencia = dias_completos.merge(referencia, on="lead_time_dias", how="left")
99         precio_global = ruta["precio_ponderado"].sum() / ruta["n_observaciones_precio"].
sum()
100         referencia["precio_base"] = referencia["precio_base"].interpolate().bfill().ffill
().fillna(precio_global)
101         return referencia, tarifa_maxima_historica
102
103         referencia_precio_train, tarifa_max_train = crear_referencia_precio(
df_precios_train_pd)
104         referencia_precio_val, tarifa_max_val = crear_referencia_precio(df_precios_val_pd
)
105         referencia_precio_test, tarifa_max_test = crear_referencia_precio(
df_precios_test_pd)
106
107     def crear_perfil_demanda(df_spark, origen="MADRID", destino="BARCELONA",
demanda_media_objetivo=3.0):
108         ruta = df_spark.filter((col(COL_ORIGEN) == origen) & (col(COL_DESTINO) == destino
))
109         actividad = ruta.groupBy("fecha_salida_dt", "lead_time_dias").agg(countDistinct(
COL_FECHA_COMPRA).alias("n_snapshots"))
110         perfil = actividad.groupBy("lead_time_dias").agg(avg("n_snapshots").alias("
actividad_media")).toPandas()
111         dias_completos = pd.DataFrame({"lead_time_dias": np.arange(1, MAX_DIAS + 1)})
112         perfil = dias_completos.merge(perfil, on="lead_time_dias", how="left")
113         perfil["actividad_media"] = perfil["actividad_media"].interpolate().bfill().ffill
().fillna(0.0)
114         perfil["urgencia"] = np.exp(-perfil["lead_time_dias"] / 15.0)
115         perfil["demanda_base"] = perfil["actividad_media"] * perfil["urgencia"]
116         media_demanda_base = perfil["demanda_base"].mean()
117         perfil["demanda_media"] = (perfil["demanda_base"] / media_demanda_base) *
demanda_media_objetivo
118         return perfil[["lead_time_dias", "demanda_media"]]
119

```

```
120     perfil_demanda_train = crear_perfil_demanda(df_train_raw)
121     perfil_demanda_val = crear_perfil_demanda(df_val_raw)
122     perfil_demanda_test = crear_perfil_demanda(df_test_raw)
123
124     ESCALA_PRECIO_TRAIN = float(tarifa_max_train)
125     total_rutas = df_clean.select(COL_ORIGEN, COL_DESTINO).dropDuplicates().count()
126
```

Listing 2: Carga distribuida y particionado cronológico en PySpark

3. Anexo C: Entorno de Simulación (Gymnasium) y DP

Construcción íntegra de la clase de entorno logístico, inyectando Poisson, elasticidad y el cálculo del Oráculo de Programación Dinámica.

```

1      class RenfePricingEnv(gym.Env):
2          metadata = {"render_modes": []}
3
4          def __init__(self, referencia_precios, perfil_demanda, tarifa_maxima=300.0,
5              escala_precio=None, demanda_total_escenario=1.0, elasticidad=-0.8, capacidad=100,
6              max_dias=30, penalizacion_vacios=0.10, potencial_dp=None, coef_reward_shaping=0.0):
7              super().__init__()
8              self.referencia_precios = referencia_precios.copy()
9              self.perfil_demanda = perfil_demanda.copy()
10             self.tarifa_maxima = float(tarifa_maxima)
11             self.escala_precio = float(escala_precio if escala_precio is not None else
12                 tarifa_maxima)
13             self.demanda_total_escenario = float(demanda_total_escenario)
14             self.elasticidad = float(elasticidad)
15             self.capacidad = int(capacidad)
16             self.max_dias = int(max_dias)
17             self.penalizacion_vacios = float(penalizacion_vacios)
18             self.potencial_dp = potencial_dp
19             self.coef_reward_shaping = float(coef_reward_shaping)
20
21             self.multiplicadores = np.array([0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4],
22                 dtype=np.float32)
23             self.action_space = spaces.Discrete(len(self.multiplicadores))
24             self.observation_space = spaces.Box(low=np.zeros(3, dtype=np.float32), high=np.
25                 ones(3, dtype=np.float32), dtype=np.float32)
26
27             self.precio_por_dia = {int(dia): float(precio) for dia, precio in zip(self.
28                 referencia_precios["lead_time_dias"], self.referencia_precios["precio_base"])}
29             self.precio_fallback = float(self.referencia_precios["precio_base"].median())
30             self.demanda_por_dia = {int(dia): float(demanda) for dia, demanda in zip(self.
31                 perfil_demanda["lead_time_dias"], self.perfil_demanda["demanda_media"])}
32             self.factor_demanda = self.demanda_total_escenario
33             self.dias_restantes = self.max_dias
34             self.asientos_libres = self.capacidad
35
36             def precio_base(self, dias_restantes):
37                 return float(self.precio_por_dia.get(int(dias_restantes), self.precio_fallback))
38
39             def demanda_esperada(self, dias_restantes, action):
40                 precio_base = self.precio_base(dias_restantes)
41                 multiplicador = float(self.multiplicadores[action])
42                 precio_final = min(precio_base * multiplicador, self.tarifa_maxima)
43                 lambda_historica = self.demanda_por_dia.get(int(dias_restantes), 0.0)
44                 lambda_base = lambda_historica * self.factor_demanda
45                 ratio_precio = (precio_final / max(precio_base, 1e-6))
46                 factor_precio = (ratio_precio ** self.elasticidad)
47                 lambda_demanda = lambda_base * factor_precio
48                 return max(float(lambda_demanda), 0.0)
49
50             def get_observacion(self):
51                 precio_base_actual = self.precio_base(self.dias_restantes)
52                 precio_base_norm = (precio_base_actual / self.escala_precio)
53                 precio_base_norm = float(np.clip(precio_base_norm, 0.0, 1.0))
54                 ocupacion = (1.0 - (self.asientos_libres / self.capacidad))
55                 return np.array([self.dias_restantes / float(self.max_dias), ocupacion,
56                     precio_base_norm], dtype=np.float32)
57
58             def reset(self, seed=None, options=None):
59                 super().reset(seed=seed)
60                 self.dias_restantes = self.max_dias
61                 self.asientos_libres = self.capacidad
62                 return self.get_observacion(), {}
63
64             def valor_potencial(self, dias_restantes, asientos_libres):
65                 if self.potencial_dp is None: return 0.0

```

```

58     dias = int(np.clip(dias_restantes, 0, self.max_dias))
59     asientos = int(np.clip(asientos_libres, 0, self.capacidad))
60     return float(self.potencial_dp[dias, asientos])
61
62     def step(self, action):
63         action = int(action)
64         potencial_antes = self.valor_potencial(self.dias_restantes, self.asientos_libres)
65         precio_base = self.precio_base(self.dias_restantes)
66         multiplicador = float(self.multiplicadores[action])
67         precio_final = min(precio_base * multiplicador, self.tarifa_maxima)
68         lambda_demanda = self.demanda_esperada(self.dias_restantes, action)
69         demanda_real = int(self.np_random.poisson(lambda_demanda))
70         entradas_vendidas = min(demanda_real, self.asientos_libres)
71
72         self.asientos_libres -= entradas_vendidas
73         ingreso_paso = float(entradas_vendidas * precio_final)
74         self.dias_restantes -= 1
75
76         terminated = bool(self.dias_restantes <= 0 or self.asientos_libres <= 0)
77         penalizacion = 0.0
78         if terminated and self.asientos_libres > 0:
79             penalizacion = float(self.asientos_libres * self.precio_fallback * self.
penalizacion_vacios)
80
81         reward_euros = (ingreso_paso - penalizacion)
82         potencial_despues = self.valor_potencial(self.dias_restantes, self.
asientos_libres)
83         reward_shaping_euros = self.coef_reward_shaping * (potencial_despues -
potencial_antes)
84         reward_total_euros = reward_euros + reward_shaping_euros
85         reward = reward_total_euros / self.escala_precio
86
87         info = {"ingreso_paso": ingreso_paso, "penalizacion_final": penalizacion}
88         return self.get_observacion(), float(reward), terminated, False, info
89
90         env_check = RenfePricingEnv(
91             referencia_precios=referencia_precio_train,
92             perfil_demanda=perfil_demanda_train,
93             tarifa_maxima=tarifa_max_train,
94             escala_precio=ESCALA_PRECIO_TRAIN,
95             demanda_total_escenario=FACTOR_DEMANDA_NOMINAL,
96             elasticidad=ELASTICIDAD_NOMINAL,
97             capacidad=CAPACIDAD,
98             max_dias=MAX_DIAS,
99             penalizacion_vacios=PENALIZACION_VACIOS,
100         )
101         check_env(env_check, warn=True)
102         env_check.close()
103
104         def calcular_optimo_dp(entorno):
105             max_dias = entorno.max_dias
106             capacidad = entorno.capacidad
107             acciones = entorno.multiplicadores
108             V = np.zeros((max_dias + 1, capacidad + 1), dtype=np.float64)
109             politica = np.zeros((max_dias + 1, capacidad + 1), dtype=np.int32)
110
111             for c in range(capacidad + 1):
112                 V[0, c] = -c * entorno.precio_fallback * entorno.penalizacion_vacios
113
114             for d in range(1, max_dias + 1):
115                 for c in range(capacidad + 1):
116                     if c == 0:
117                         V[d, c] = 0.0
118                     politica[d, c] = 0
119                     continue
120                 valores = np.zeros(len(acciones))
121                 for a in range(len(acciones)):
122                     precio_base = entorno.precio_base(d)
123                     multiplicador = float(acciones[a])
124                     precio = min(precio_base * multiplicador, entorno.tarifa_maxima)
125                     lam = entorno.demanda_esperada(d, a)
126                     valor = 0.0
127                     if lam > 0:

```

```
128     dem_array = np.arange(c)
129     probs = poisson.pmf(dem_array, lam)
130     valores_futuros = V[d - 1, c - dem_array]
131     valor += np.sum(probs * (dem_array * precio + valores_futuros))
132     prob_agotar = poisson.sf(c - 1, lam)
133     valor += prob_agotar * (c * precio + V[d - 1, 0])
134     else:
135     valor += V[d - 1, c]
136     valores[a] = valor
137     politica[d, c] = int(np.argmax(valores))
138     V[d, c] = float(np.max(valores))
139     return V, politica
140
```

Listing 3: Implementación core del MDP y Óptimo Teórico

4. Anexo D: Baseline Machine Learning Tradicional

Generación de datos sintéticos y entrenamiento del modelo supervisor (Random Forest) para contrastar su miopía frente al RL.

```

1  def crear_dataset_rf_real(df_spark, n_muestras=50000, seed=SEED):
2      rng = np.random.default_rng(seed)
3      filas = []
4      rutas = df_spark.select(COL_ORIGEN, COL_DESTINO, "lead_time_dias",
5                              COL_TIPO_VEHICULO, COL_CLASE, COL_TARIFA, COL_PRECIO).distinct().toPandas()
6
7      for _ in range(n_muestras):
8          fila = rutas.sample(1).iloc[0]
9          dias = int(fila["lead_time_dias"])
10         origen = fila[COL_ORIGEN]
11         destino = fila[COL_DESTINO]
12         precio_base = float(fila[COL_PRECIO])
13         asientos = int(rng.integers(20, CAPACIDAD + 1))
14         multiplicador = float(rng.choice(np.array([0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2,
15             1.3, 1.4])))
16         precio_final = min(precio_base * multiplicador, 300.0)
17
18         demanda_latente = max(1.0, (asientos * 0.55) * (1.0 / max(dias, 1)) *
19             multiplicador)
20         demanda_sintetica = int(rng.poisson(demanda_latente))
21         ventas_sinteticas = min(demanda_sintetica, asientos)
22
23         filas.append({
24             "dias_restantes": dias,
25             "asientos_libres": asientos,
26             "multiplicador": multiplicador,
27             "precio_base": precio_base,
28             "precio_final": precio_final,
29             "ventas_sinteticas": ventas_sinteticas,
30             "origen": origen,
31             "destino": destino,
32         })
33     return pd.DataFrame(filas)
34
35     df_rf_real = crear_dataset_rf_real(df_train_raw, n_muestras=50_000, seed=SEED)
36     COLUMNAS_RF = ["dias_restantes", "asientos_libres", "multiplicador", "precio_base"]
37
38     X_rf = df_rf_real[COLUMNAS_RF]
39     y_rf = df_rf_real["ventas_sinteticas"]
40
41     modelo_rf = RandomForestRegressor(n_estimators=400, min_samples_leaf=4,
42         random_state=SEED, n_jobs=-1)
43     modelo_rf.fit(X_rf, y_rf)
44
45     CACHE_ACCIONES_RF = {}
46
47     def elegir_accion_rf(estado, entorno):
48         estado_array = np.asarray(estado, dtype=np.float32)
49         dias = int(np.round(estado_array[0] * entorno.max_dias))
50         ocupacion = float(estado_array[1])
51         asientos = int(np.round((1.0 - ocupacion) * entorno.capacidad))
52
53         if dias <= 0 or asientos <= 0:
54             return 4
55
56         precio_base = entorno.precio_base(dias)
57         clave = (dias, asientos, round(precio_base, 2))
58         if clave in CACHE_ACCIONES_RF:
59             return CACHE_ACCIONES_RF[clave]
60
61         multiplicadores = entorno.multiplicadores
62         candidatos = pd.DataFrame({
63             "dias_restantes": np.full(len(multiplicadores), dias),
64             "asientos_libres": np.full(len(multiplicadores), asientos),
65             "multiplicador": multiplicadores,
66             "precio_base": np.full(len(multiplicadores), precio_base)

```

```
63     })
64
65     demanda_predicha = modelo_rf.predict(candidatos)
66     ventas_predichas = np.minimum(np.maximum(demanda_predicha, 0), asientos)
67     precios_reales = np.minimum(precio_base * multiplicadores, entorno.tarifa_maxima)
68     ingresos_esperados = ventas_predichas * precios_reales
69     mejor_accion = int(np.argmax(ingresos_esperados))
70
71     CACHE_ACCIONES_RF[clave] = mejor_accion
72     return mejor_accion
73
```

Listing 4: Entrenamiento Baseline Supervisado

5. Anexo E: Entrenamiento del Agente Autónomo PPO

Definición de hiperparámetros y orquestación del ciclo de entrenamiento, inyectando Callbacks entrópicos para evitar el colapso del modelo.

```

1      from stable_baselines3.common.callbacks import BaseCallback, CallbackList
2      import torch as th
3
4      class EntropiaPPOCallback(BaseCallback):
5      def __init__(self, estados_diagnostico, frecuencia=10_000, verbose=0):
6      super().__init__(verbose)
7      self.estados_diagnostico = estados_diagnostico
8      self.frecuencia = frecuencia
9      self.historial = []
10
11     def _on_step(self):
12     if self.num_timesteps % self.frecuencia == 0:
13     obs = th.as_tensor(self.estados_diagnostico, dtype=th.float32, device=self.model.
device)
14     with th.no_grad():
15     distribucion = self.model.policy.get_distribution(obs)
16     entropia = float(distribucion.entropy().mean().cpu().item())
17     self.historial.append({"timesteps": int(self.num_timesteps), "entropia_media":
entropia})
18     return True
19
20     def entrenar_ppo(semilla, configuracion):
21     set_random_seed(semilla)
22     nombre = configuracion["nombre"]
23     carpeta_modelo = RUTA_MODELOS / f"ppo_revisado_{nombre}_seed_{semilla}"
24     carpeta_logs = RUTA_LOGS / f"ppo_revisado_{nombre}_seed_{semilla}"
25
26     env_train = crear_entorno_train(configuracion["coef_reward_shaping"])
27     env_val = crear_entorno_validacion()
28
29     try:
30     callback_eval = EvalCallback(
31     env_val, best_model_save_path=str(carpeta_modelo), log_path=str(carpeta_logs),
32     eval_freq=10_000, n_eval_episodes=50, deterministic=True, render=False
33     )
34
35     callback_entropia = EntropiaPPOCallback(ESTADOS_DIAGNOSTICO, frecuencia=10_000)
36     callback = CallbackList([callback_eval, callback_entropia])
37
38     modelo = PPO(
39     "MlpPolicy", env_train,
40     learning_rate=configuracion["learning_rate"],
41     n_steps=configuracion["n_steps"],
42     batch_size=configuracion["batch_size"],
43     gamma=configuracion["gamma"],
44     gae_lambda=configuracion["gae_lambda"],
45     policy_kwargs={"net_arch": {"pi": [128, 128], "vf": [128, 128]}},
46     seed=semilla,
47     ent_coef=configuracion["ent_coef"],
48     verbose=0,
49     tensorboard_log=str(RUTA_LOGS),
50     )
51
52     modelo.learn(total_timesteps=TIMESTEPS_PPO, callback=callback, tb_log_name=f"PP0_
{nombre}_{semilla}")
53
54     ruta_mejor = carpeta_modelo / "best_model.zip"
55     mejor_modelo = PPO.load(str(ruta_mejor))
56
57     return mejor_modelo, callback_entropia.historial
58     finally:
59     env_train.close()
60     env_val.close()

```

61

Listing 5: Ciclo de aprendizaje de PPO

6. Anexo F: Evaluación de Negocio y Simulación Monte Carlo

Algoritmo de inferencia para testear todas las estrategias (Base, Heurística, RF y PPO) frente a los datos del set Test en miles de episodios iterativos.

```

1     ESCENARIOS_DEMANDA = {"Baja": 0.80, "Media": 1.00, "Alta": 1.20}
2     ESCENARIOS_ELASTICIDAD = {"Menos sensible": -0.4, "Nominal": -1.2, "Más sensible"
: -1.8}
3     N_EVALUACIONES = 1_000
4
5     def accion_heuristica(estado, entorno):
6         dias = int(np.clip(np.round(estado[0] * entorno.max_dias), 0, entorno.max_dias))
7         ocupacion = float(np.clip(estado[1], 0.0, 1.0))
8         if dias <= 10 and ocupacion >= 0.80:
9             return 8
10        if dias >= 20 and ocupacion <= 0.40:
11            return 2
12        return 4
13
14    def ejecutar_episodio(estrategia, entorno, semilla, modelo_ppo=None, politica_dp=
None):
15        estado, _ = entorno.reset(seed=semilla)
16        terminado = False
17        ingreso_total = 0.0
18        penalizacion_total = 0.0
19        reward_total = 0.0
20        acciones = []
21
22        while not terminado:
23            dias = int(np.clip(np.round(estado[0] * entorno.max_dias), 0, entorno.max_dias))
24            asientos = int(np.clip(np.round((1.0 - float(estado[1])) * entorno.capacidad), 0,
entorno.capacidad))
25
26            if estrategia == "Política base (1.0x)":
27                accion = 4
28            elif estrategia == "Heurística":
29                accion = accion_heuristica(estado, entorno)
30            elif estrategia == "RF sintético miope":
31                accion = elegir_accion_rf(estado, entorno)
32            elif estrategia == "DP oracle / techo teórico":
33                accion = int(politica_dp[dias, asientos])
34            elif estrategia == "PPO":
35                accion_predicha, _ = modelo_ppo.predict(estado, deterministic=True)
36                accion = int(np.asarray(accion_predicha).item())
37
38            acciones.append(accion)
39            estado, reward, terminado, truncado, info = entorno.step(accion)
40
41            ingreso_total += float(info["ingreso_paso"])
42            penalizacion_total += float(info["penalizacion_final"])
43            reward_total += float(reward)
44
45            objetivo_total = ingreso_total - penalizacion_total
46            ocupacion_final_pct = 100.0 * (entorno.capacidad - entorno.asientos_libres) /
entorno.capacidad
47            multiplicador_medio = float(np.mean([entorno.multiplicadores[a] for a in acciones
])) if acciones else 1.0
48
49        return {
50            "ingreso_total": ingreso_total,
51            "penalizacion_total": penalizacion_total,
52            "objetivo_total": objetivo_total,
53            "reward_total": reward_total,
54            "ocupacion_final_pct": ocupacion_final_pct,
55            "multiplicador_medio": multiplicador_medio,
56        }
57
58    # Bootstrapping para Intervalos de Confianza al 95%
59    def bootstrap_ic_episodios(diferencias, n_boot=3_000, seed=SEED):
60        diferencias = np.asarray(diferencias, dtype=float)

```

```
61     rng = np.random.default_rng(seed)
62     medias = np.empty(n_boot, dtype=float)
63     for i in range(n_boot):
64         muestra = rng.choice(diferencias, size=len(diferencias), replace=True)
65         medias[i] = np.mean(muestra)
66     return tuple(np.percentile(medias, [2.5, 97.5]))
67
```

Listing 6: Motor de Evaluación y Extracción de Métricas

7. Anexo G: Reconstrucción e Interpretación Económica PPO

Diagnóstico visual y matricial de la política dinámica aprendida por el agente PPO.

```

1  politica_ppo = np.zeros((MAX_DIAS + 1, CAPACIDAD + 1), dtype=int)
2  estados_batch = []
3  coordenadas = []
4
5  for dias in range(1, MAX_DIAS + 1):
6      precio_base = env_dp.precio_base(dias)
7      precio_norm = float(np.clip(precio_base / ESCALA_PRECIO_TRAIN, 0.0, 1.0))
8      for asientos_libres in range(1, CAPACIDAD + 1):
9          ocupacion = 1.0 - (asientos_libres / CAPACIDAD)
10         estados_batch.append([dias / MAX_DIAS, ocupacion, precio_norm])
11         coordenadas.append((dias, asientos_libres))
12
13     estados_array = np.array(estados_batch, dtype=np.float32)
14     acciones_batch, _ = modelo_ppo_best.predict(estados_array, deterministic=True)
15
16     for (dias, asientos_libres), accion in zip(coordenadas, acciones_batch):
17         politica_ppo[dias, asientos_libres] = int(accion)
18
19     politica_util = politica_ppo[1:, 1:]
20     acciones_ppo, frecuencias_ppo = np.unique(politica_util, return_counts=True)
21
22     df_distribucion_ppo = pd.DataFrame({
23         "accion": acciones_ppo,
24         "multiplicador": [float(env_dp.multiplicadores[a]) for a in acciones_ppo],
25         "estados": frecuencias_ppo,
26         "porcentaje": (frecuencias_ppo / politica_util.size * 100.0)
27     })
28
29     import seaborn as sns
30     import matplotlib.pyplot as plt
31
32     plt.figure(figsize=(12, 7))
33     plt.imshow(politica_util[:-1, :], aspect="auto")
34     plt.xlabel("Asientos libres")
35     plt.ylabel("Días restantes")
36     plt.title("Mapa de política PPO seleccionada")
37     plt.colorbar(label="Índice de acción")
38     plt.show()
39

```

Listing 7: Disección de la Matriz PPO y Estadísticas

8. Anexo H: MLOps y Exportación Serializada a Web

Preparación del artefacto productivo: desacoplamiento de la red neuronal hacia un microservicio estructurado en JSON consumible por el Frontend.

```

1     cerebro_multiruta = {}
2
3     for origen, destino in rutas_disponibles:
4         try:
5             referencia_ruta, tarifa_max_ruta = crear_referencia_precio(df_completo_desarrollo
, origen=origen, destino=destino)
6
7             precios_base_diarios = {
8                 str(int(fila["lead_time_dias"])): float(fila["precio_base"])
9                 for _, fila in referencia_ruta.iterrows()
10            }
11
12            estados_export = []
13            coords_export = []
14
15            for dias in range(0, MAX_DIAS + 1):
16                precio_base_dia = precios_base_diarios.get(
17                    str(dias), float(referencia_precio_train[referencia_precio_train["lead_time_dias"
] == dias]["precio_base"].iloc[0])
18                )
19                precio_norm = float(np.clip(precio_base_dia / ESCALA_PRECIO_TRAIN, 0.0, 1.0))
20
21                for asientos_libres in range(0, CAPACIDAD + 1):
22                    ocupacion = 1.0 - (asientos_libres / CAPACIDAD)
23                    estados_export.append([dias / MAX_DIAS, ocupacion, precio_norm])
24                    coords_export.append(f"{dias}_{asientos_libres}")
25
26                estados_array_exp = np.array(estados_export, dtype=np.float32)
27                acciones_exp, _ = modelo_ppo_best.predict(estados_array_exp, deterministic=True)
28
29                matriz_ruta = {coord: int(acc) for coord, acc in zip(coords_export, acciones_exp)}
30            }
31
32            clave_ruta = f"{origen}_{destino}".upper().replace(" ", "_")
33
34            cerebro_multiruta[clave_ruta] = {
35                "politica": matriz_ruta,
36                "precios_base": precios_base_diarios,
37                "tarifa_maxima": float(tarifa_max_ruta),
38                "escala_precio": float(ESCALA_PRECIO_TRAIN),
39            }
40
41        except (ValueError, KeyError, IndexError) as error:
42            print(f"Ruta no exportada {origen}-{destino}: {error}")
43
44        with open(ruta_export / "cerebro.js", "w", encoding="utf-8") as archivo:
45            archivo.write("const CEREBRO_IA = " + json.dumps(cerebro_multiruta, ensure_ascii=
False) + ";")

```

Listing 8: Serialización $\mathcal{O}(1)$ para el Frontend interactivo

9. Anexo I: Arquitectura Guardrails y Alertas en Vanilla JS

Mecanismo determinista de gobernanza integrado en el Dashboard web para truncar inferencias de la IA y emitir notificaciones visuales auditables.

```
1      // Intercepcion y evaluacion determinista de la politica DRL
2      const pCrudo = pBase * mult;
3      const veto = pCrudo > max;
4      const pFinal = veto ? max : pCrudo; // Truncamiento corporativo de seguridad
5
6      // Manipulacion del DOM para alertas visuales al analista
7      if (veto) {
8          D.kpiVarTexto.innerHTML = '<span class="text-rose-600 font-bold">LIMITE
9      APLICADO</span>';
10         D.bannerGob.classList.remove("hidden");
11     } else {
12         D.bannerGob.classList.add("hidden");
13     }
```

Listing 9: Intercepción de seguridad en el Frontend